

ПРАКТИЧЕСКАЯ РАБОТА №6

«Алгоритмы поиска»

Часть 6.2. Поиск образца в тексте

Вариант 4

Цель: освоить приёмы реализации алгоритмов поиска образца в тексте.

1. Задание 1

1.1. Формулировка задачи

Дано предложение, состоящее из слов, разделённых знаками препинания. Определить, сколько раз в предложение входит первое слово.

1.2. Математическая модель решения

Предложение разбивается на слова: слово — максимальная последовательность символов, не входящих в множество разделителей " , . ; : ! ? - () \ " ' . Затем каждое слово сравнивается с первым целиком (поэтому «котёнок» не считается вхождением слова «кот»), счётчик учитывает и само первое слово. Сложность $O(n)$, где n — длина предложения.

1.3. Код программы

```
1 // Разбивает предложение на слова. Слово - непрерывная последовательность
2 // символов, не являющихся разделителями.
3 std::vector<std::string> splitWords(const std::string& s) {
4     std::vector<std::string> words;
5     std::string w;
6     for (char c : s) {
7         if (SEPARATORS.find(c) == std::string::npos) {
8             w += c;
9         } else if (!w.empty()) {
10             words.push_back(w);
11             w.clear();
12         }
13     }
14     if (!w.empty()) words.push_back(w);
15     return words;
16 }
```

```
1 // Предусловие: s - предложение из слов, разделенных знаками препинания.
2 // Постусловие: возвращено число вхождений первого слова в предложение
3 // (с учетом его самого и регистра букв); 0, если слов нет.
4 int countFirstWord(const std::string& s) {
5     std::vector<std::string> words = splitWords(s);
6     int count = 0;
7     for (const std::string& w : words)
8         if (w == words[0]) count++;
9 }
```

```

9     return count;
10  }
```

1.4. Результаты тестирования

Таблица 1.1 — Тесты задания 1

Предложение	Результат
кот, кот; собака: кот! мышь, котёнок.	3
Мама мыла раму, мама - молодец.	1 (регистр учитывается)
один	1
раз, два, три, раз-раз, раз!	4
пустая строка	0

2. Задание 2

2.1. Формулировка задачи

Проверка на плагиат. Используя алгоритм Рабина–Карпа, проверить, входит ли подстрока проверяемого текста в другой текст. Подстроками считаются все фрагменты проверяемого текста длины k ; найденные в другом тексте фрагменты отмечаются как заимствованные.

2.2. Математическая модель решения

Полиномиальный хеш строки $c_1 c_2 \dots c_m$ (основание $b = 256$, простой модуль $Q = 10^9 + 7$):

$$H = (c_1 b^{m-1} + c_2 b^{m-2} + \dots + c_m b^0) \bmod Q$$

Хеш образца сравнивается с хешем подстроки текста. Неравенство хешей гарантирует, что образца в этой позиции нет; при равенстве возможно ложное совпадение (коллизия), поэтому строки сравниваются посимвольно.

Кольцевой (скользящий) хеш — хеш подстроки, сдвинутой на 1 вправо, вычисляется из предыдущего за $O(1)$:

$$H = ((H_p - C_p b^{m-1})b + C_n) \bmod Q$$

где H_p — предыдущий хеш, C_p — ушедший символ, C_n — новый символ.

Поиск одиночного образца: хеши всех $n - m + 1$ подстрок сравниваются с хешем образца. Сложность в среднем $O(n + m)$, в худшем (много совпадений хешей) $O(nm)$.

Поиск множества образцов одинаковой длины (применяется для проверки на плагиат): хеши всех фрагментов длины k проверяемого текста заносятся в таблицу «хеш $\rightarrow$ позиции»; затем окно длины k проходит по другому тексту, и если хеш окна есть в таблице, фрагменты с этим хешем сверяются посимвольно. Время в среднем $O(n + m)$ независимо от числа образцов.

Пример. Текст `abracadabra` ($n = 11$), $m = 3$, для наглядности $Q = 13$. Коды символов по модулю 13: $a = 6$, $b = 7$, $c = 8$, $d = 9$, $r = 10$; $b \bmod 13 = 9$, $b^2 \bmod 13 = 3$, т. е. $H(xyz) = (3x + 9y + z) \bmod 13$.

Таблица 2.1 — Хеши подстрок текста

i	0	1	2	3	4	5	6	7	8
подстрока	abr	bra	rac	aca	cad	ada	dab	abr	bra
H	0	0	1	5	9	1	10	0	0

Успешный поиск `cad`: хеш образца 9. Хеши сравниваются при $i = 0..4$ (5 сравнений), при $i = 4$ совпадение, проверка 3 символов — итого **8 сравнений**, индекс 4.

Безуспешный поиск `rab`: хеш образца 0. Сравниваются все 9 хешей; при $i = 0, 1, 7, 8$ — ложные совпадения, каждое отсекается на первом символе (4 сравнения) — итого **13 сравнений**, результат —1.

При $Q = 10^9 + 7$ ложные совпадения практически исключены.

2.3. Код программы

```

1 // Полиномиальный хеш m символов строки s, начиная с позиции start:
2 // H = (c1*b^(m-1) + c2*b^(m-2) + ... + cm*b^0) mod Q
3 long long polyHash(const std::string& s, int start, int m, long long q) {
4     long long h = 0;
5     for (int i = start; i < start + m; i++)
6         h = (h * B + (unsigned char)s[i]) % q;
7     return h;
8 }
```

```

1 // b^(m-1) mod Q - вес первого символа подстроки
2 long long topWeight(int m, long long q) {
3     long long w = 1;
4     for (int i = 0; i < m - 1; i++) w = w * B % q;
5     return w;
6 }
```

```

1 // Кольцевой (скользящий) хеш - хеш подстроки, сдвинутой на 1 вправо:
2 // H = ((Hr - Cr*b^(m-1)) * b + Cn) mod Q,
3 // Hr - предыдущий хеш, Cr - ушедший символ, Cn - новый символ, w = b^(m-1)
```

```

4 long long rollHash(long long hp, char cp, char cn, long long w, long long q) {
5     hp = (hp - (unsigned char)cp * w % q + q) % q;
6     return (hp * B + (unsigned char)cn) % q;
7 }

```

```

1 // Поиск Рабина-Карпа одиночного образца.
2 // Предусловие: q - простое число.
3 // Постусловие: возвращен индекс первого вхождения sub в s или -1;
4 // к cmp прибавлено число выполненных сравнений (хешей и символов).
5 int rabinKarp(const std::string& s, const std::string& sub, long long& cmp,
6               long long q = Q) {
7     int n = s.size(), m = sub.size();
8     if (m == 0) return 0;
9     if (m > n) return -1;
10
11     long long w = topWeight(m, q);
12     long long hsub = polyHash(sub, 0, m, q); // хеш образца
13     long long hs = polyHash(s, 0, m, q);     // хеш подстроки текста
14     for (int i = 0; i ≤ n - m; i++) {
15         cmp++;
16         if (hs == hsub) { // хеши совпали - проверяем символы
17             int j = 0;
18             while (j < m) {
19                 cmp++;
20                 if (s[i + j] ≠ sub[j]) break;
21                 j++;
22             }
23             if (j == m) return i;
24         }
25         if (i < n - m) hs = rollHash(hs, s[i], s[i + m], w, q);
26     }
27     return -1;
28 }

```

```

1 // Проверка на плагиат - поиск Рабина-Карпа множества образцов одинаковой
2 // длины. Образцы - все фрагменты проверяемого текста длиной k байт.
3 // Предусловие: k > 0.
4 // Постусловие: borrowed[i] = true, если i-й байт проверяемого текста входит
5 // во фрагмент длины k, который встречается в другом тексте.
6 std::vector<bool> plagiarism(const std::string& checked,
7                              const std::string& source, int k) {
8     int n = checked.size(), m = source.size();
9     std::vector<bool> borrowed(n, false);
10     if (k > n || k > m) return borrowed;
11     long long w = topWeight(k, Q);
12
13     // 1. Хеши всех образцов: хеш → позиции образцов в проверяемом тексте
14     std::map<long long, std::vector<int>> hsubs;
15     long long h = polyHash(checked, 0, k, Q);
16     for (int i = 0; i + k ≤ n; i++) {
17         hsubs[h].push_back(i);
18         if (i + k < n) h = rollHash(h, checked[i], checked[i + k], w, Q);
19     }
20 }

```

```

19     }
20
21     // 2. Окно длины k проходит по другому тексту
22     long long hs = polyHash(source, 0, k, Q);
23     for (int i = 0; i + k ≤ m; i++) {
24         auto it = hsubs.find(hs);
25         if (it ≠ hsubs.end()) // хеш есть среди образцов - сверяем символы
26             for (int p : it→second)
27                 if (source.compare(i, k, checked, p, k) == 0)
28                     for (int j = p; j < p + k; j++) borrowed[j] = true;
29         if (i + k < m) hs = rollHash(hs, source[i], source[i + k], w, Q);
30     }
31     return borrowed;
32 }

```

```

1 // Длина символа UTF-8, начинающегося с байта c (русская буква - 2 байта)
2 int charLen(unsigned char c) {
3     if (c < 0x80) return 1;
4     if (c < 0xE0) return 2;
5     if (c < 0xF0) return 3;
6     return 4;
7 }

```

```

1 // Выводит проверяемый текст, заключая заимствованные фрагменты в [ ],
2 // и число заимствованных символов. Символ заимствован, если заимствованы
3 // все его байты.
4 void printPlagiarism(const std::string& checked,
5                     const std::vector<bool>& borrowed) {
6     int total = 0, copied = 0;
7     bool prev = false;
8     std::string out;
9     for (int i = 0; i < (int)checked.size(); total++) {
10         int len = charLen(checked[i]);
11         bool b = true;
12         for (int j = i; j < i + len; j++) b = b && borrowed[j];
13         if (b && !prev) out += '[';
14         if (!b && prev) out += ']';
15         out += checked.substr(i, len);
16         if (b) copied++;
17         prev = b;
18         i += len;
19     }
20     if (prev) out += ']';
21     std::cout << " " << out << "\n Заимствовано символов: " << copied
22               << " из " << total << "\n";
23 }

```

2.4. Результаты тестирования

Проверка на плагиат, $k = 30$ байт (≈ 15 русских букв), заимствованные фрагменты выделены []:

- 1 Другой текст: "Хеш-таблица хранит пары ключ-значение. Поиск в ней
- 2 выполняется в среднем за константное время. Коллизии разрешаются методом
- 3 цепочек или открытой адресацией."
- 4
- 5 [Поиск в ней выполняется в среднем за константное время], это удобно[. Коллизии разрешают методом цепочек].
- 6 Заимствовано символов: 90 из 103
- 7 Дерево - это связный граф без циклов.
- 8 Заимствовано символов: 0 из 37
- 9 [Хеш-таблица хранит пары ключ-значение.]
- 10 Заимствовано символов: 38 из 38

Таблица 2.2 — Тесты поиска одиночного образца ($Q = 10^9 + 7$)

Тест	n	m	Индекс	Сравнений
hello world / world	11	5	6	12
hello world / word	11	4	−1	8
образец длиннее текста	3	4	−1	0
пустой образец	3	0	0	0
большой текст, образец из середины	10^6	10	500 000	500 011
большой текст, образца нет	10^6	10	−1	999 991
большой образец из текста	10^6	50 000	900 000	950 001
большой образец, нет в тексте	10^6	50 000	−1	950 001

Большой текст — 10^6 случайных латинских букв.

2.5. Практическая сложность

Безуспешный поиск одиночного образца в случайном тексте, $Q = 10^9 + 7$:

Таблица 2.3 — Зависимость от длины текста и образца

n	m	Сравнений	Время, мс
10^4	10	9 991	0,055
10^4	100	9 901	0,055
10^4	1000	9 001	0,056
10^5	10	99 991	0,549
10^5	100	99 901	0,546
10^5	1000	99 001	0,561
10^6	10	999 991	5,469
10^6	100	999 901	5,505
10^6	1000	999 001	5,482

Число сравнений равно $n - m + 1$, время растёт линейно по n и почти не зависит от m . При $Q = 13$ ($n = 10^6$, $m = 100$) сравнений 1 079 702 вместо 999 901: добавляются посимвольные проверки ложных совпадений хешей.

3. Вывод

Реализованы подсчёт вхождений первого слова в предложение и проверка на плагиат алгоритмом Рабина–Карпа (полиномиальный и кольцевой хеш, поиск множества образцов одинаковой длины). Тесты подтвердили корректность успешного и безуспешного поиска, замеры — линейную зависимость времени от длины текста $O(n + m)$ при большом простом модуле Q . Цель работы достигнута.